

Blockchain: Exam's Possible Questions

Question 1

What are the key properties of a cryptographically secure hash function?

A cryptographically secure hash function must satisfy the following three key properties: **collision-resistance**, **hiding**, **puzzle-friendliness**.

1. A hash function H is said to be collision resistant if it is infeasible to find two values x and y , such that $x \neq y$ and $H(x) = H(y)$. It's important to note that actually we suspect that a hash function is collision resistant, but there are no hash functions proven to be collision-resistant. The cryptographic hash functions that we rely on in practice are just functions for which people have tried really really hard to find collisions and haven't yet succeeded.

2. A hash function H is hiding if, when a secret value r is chosen from a distribution with high min-entropy, then given $H(r\|x)$, it is computationally infeasible to find x , where $\|$ denotes the concatenation operator.

3. A hash function H is said to be puzzle-friendly if for every possible n -bit output value y , if k is chosen from a distribution with high min-entropy, then it is infeasible to find x such that $H(k\|x)$ equals to a target value y in time significantly less than 2^n . (Sec 1.1 From Bitcoin Book)

Question 2

Explain the Merkle-Damgård construction. What issue does it address, and how does it work?

The **Merkle-Damgård transform** is a method that allows a compression function to be used for inputs of arbitrary length. Conceptually speaking, a **compression function** takes an input of length m and produces an output of length n . The Merkle-Damgård transform allows such a function to process inputs of arbitrary length by first dividing the message into blocks of size $m - n$. Each block is then concatenated with the output of the previous compressed block (which is of length n), resulting in an input of length m for the compression function. For the first block, since there is no previous output, a fixed initialization vector (IV) of length n is used. The final block is eventually padded and the output of the last compression becomes the result of the hash function. (Sec 1.1 From Bitcoin Book)

Question 3

Suppose you have a 2024-bit string and a hash function that uses a com-

pression function with a 768-bit input and a 256-bit output. Into how many chunks should the string be divided, and what should be the length of each chunk?

Let's define i and o as the input and output lengths of the compression function respectively. Then, the length of each message chunk is given by:

$$c = i - o = 768 - 256 = 512.$$

Finally, given a message of length 2024 bits, the number of chunks is:

$$d = \left\lceil \frac{2024}{c} \right\rceil = \left\lceil \frac{2024}{512} \right\rceil = \lceil 3.953125 \rceil = 4.$$

Question 4

How does a digital signature scheme operate? Describe the two essential properties required for its security and effectiveness.

A digital signature scheme consists of three algorithms:

- $(sk, pk) := \text{generateKeys}(\text{keysize})$ generates a key pair consisting of a private key sk and a public key pk , given a key size. The private key is used to sign messages, while the public key is used by anyone who wants to verify the signature.
- $\text{sig} := \text{sign}(sk, \text{message})$ returns a signature sig for the given message using the secret key sk .
- $\text{isValid} := \text{verify}(pk, \text{message}, \text{sig})$ takes a message, a signature, and a public key as input and returns a boolean value that will be **true** if sig is a valid signature for message under public key pk , and **false** otherwise.

Moreover, for the scheme to be secure, the following must hold:

- A valid signature must verify correctly: $\text{verify}(pk, \text{message}, \text{sign}(sk, \text{message})) = \text{true}$.
- Signatures must be existentially unforgeable, meaning that it is computationally infeasible for an adversary to forge a valid signature for any message, even after observing the public key and a number of message-signature pairs. (Sec 1.3 From Bitcoin Book)

Question 5

What are the economic incentives that drive participation in the Bitcoin network?

There are two separate incentive mechanisms in Bitcoin. The first mechanism is called the **block reward**. It allows the node that creates a block

to include a coin-creation transaction in the block, which sends the newly created bitcoins to an address chosen by the node. The current reward is 3.125 BTC, and it is halved every 210,000 blocks. Based on the average block creation time of 9 to 10 minutes, this halving occurs approximately every 4 years, and as a result, the total number of bitcoins will progressively converge to 21,000,000.

The other mechanism consists of a completely voluntary **transaction fee** system. When a user creates a transaction, they can choose to make the total value of the outputs less than the total value of the inputs. In such a case, the difference is considered a transaction fee, which can be claimed by the node that creates the block including the transaction. In other words, if a node creates a block containing 100 transactions, the sum of all their transaction fees is sent to an address chosen by the node and recorded in the block. (Sec 2.4 From Bitcoin Book)

Question 6

What is the block reward in Bitcoin? How has it changed over time, and what will happen to it in the future?

The **block reward** in Bitcoin is an economic incentive mechanism that allows the node that creates a block to include a coin-creation transaction in the block, which sends the newly created bitcoins to an address chosen by the node. The current reward is 3.125 BTC, and it is halved every 210,000 blocks. Based on the average block creation time of 9 to 10 minutes, this halving occurs approximately every 4 years, and as a result, the total number of bitcoins will progressively converge to 21,000,000. (Sec 2.4 From Bitcoin Book)

Question 7

How does the proof-of-work mechanism function in Bitcoin?

Proof-of-work is a method for approximating the random selection of a node by instead selecting nodes in proportion to their computing power, a resource that ideally is not easily monopolized. In other words, it allows nodes to compete by using their computational resources, resulting in nodes being selected with a probability proportional to their relative computing power. Bitcoin achieves proof-of-work using **hash puzzles**. In order to create a block in fact, the node proposing the block must find a number, or **nonce**, such that when the nonce, the previous block hash, and the list of transactions are concatenated and hashed,

the resulting hash output falls within a target space that is very small compared to the much larger output space of the hash function. More specifically, the target space includes all values that are less than a certain target value. Thus, the nonce must satisfy the following inequality:

$$H(\text{nonce} \parallel \text{prev_hash} \parallel \text{tx} \parallel \text{tx} \parallel \dots \parallel \text{tx}) < \text{target}$$

If the hash function satisfies the puzzle-friendliness property, then the only way to succeed in solving this hash puzzle is to just try enough nonces one by one until you get lucky. That's how the system grants decentralization, since there is nobody deciding which node will be the next. Hash puzzles in Bitcoin have three important properties. First, they are difficult to compute. Second, they have a parameterizable computational cost, meaning that all nodes recalculate the target every 2016 blocks (approximately every two weeks) to ensure that the average time between successive blocks remains close to 10 minutes. Finally, it is trivial for any node to verify that the nonce of a block satisfies the target space. (Sec 2.4 From Bitcoin Book)

Question 8

What is a Coinbase transaction, and how does it differ from other transactions?

A **coinbase transaction** is a special type of transaction that creates new coins. It differs from regular transactions in several ways. First, it has only one input and one output. The input does not redeem any previous output and therefore uses a null hash pointer. The output's value is currently 3.125 BTC, but it is halved every 210,000 blocks, approximately every four years, since blocks are proposed every 9 to 10 minutes on average. As a result of this progressive halving, the total number of bitcoins will eventually converge to 21,000,000 BTC. Finally, a coinbase transaction includes a **coinbase** parameter, which is an arbitrary field that miners can fill with any data they choose. (Sec 3.4 From Bitcoin Book)

Question 9

How many decimal places does a Bitcoin have? Define what a "Satoshi" is in the context of Bitcoin.

A Bitcoin can be divided into 8 decimal places. As a result, the smallest possible value is 0.00000001 BTC, which is called 1 Satoshi. The unit has been named in collective homage to the original creator of Bitcoin, **Satoshi Nakamoto**. (Sec 2.5 From Bitcoin Book)

Question 10

What is a 51% attack? Describe the types of attacks it enables and how such attacks could be executed.

A **51% attack** refers to a scenario in a proof-of-work blockchain where a single miner or group of miners controls more than 50% of the total computing power. Such a concentration of computational resources undermines the decentralization mechanism of the blockchain.

The simplest attack that can result from a centralization of computational power is the **forking attack**, where typically, the attacker uses their dominant hashing power to make a profit by purchasing a good or service with Bitcoin. The corresponding transaction is then included in a block of the blockchain. However, as soon as the victim delivers the product or performs the service, the attacker starts extending the blockchain from a block prior to the one containing the payment transaction. By doing this, they eventually build a longer chain that excludes the transaction in question. Once their chain becomes the longest and is accepted by the network, the transaction is invalidated, allowing the attacker to obtain the good or service without actually paying for it. In any case, such an attack is detectable and would severely damage the community's trust in the system. This loss of confidence would likely lead to a collapse in the exchange rate of Bitcoin, ultimately working against the attacker's own interests.

A further possible attack is the **temporary block-withholding attack**, in which the attacker finds a valid block but chooses not to publish it immediately on top of the blockchain. Instead, they keep it private and continue mining the next block in secret. Meanwhile, other miners are still working to find the first block, unaware that it has already been found. If the attacker successfully mines a second block before the others complete the first, they can then publish both blocks at once, causing the honest miners' block to become orphaned, effectively wasting their computational effort and giving the attacker an unfair advantage.

Attacks such as **punitive forking** are also considerable; in these scenarios, a miner or mining pool that controls the majority of the hash power may decide to reject any transaction from a specific address. Since they hold the majority, this is enough to ensure that the blacklisted transactions are never confirmed. In fact, other miners are unlikely to include these transactions in their blocks, as doing so would increase the risk of their

block being orphaned, ultimately losing the associated reward. (Sec 5.5 From Bitcoin Book)

Question 11

What are Bitcoin “UTXOs”? Explain their role in tracking Bitcoin balances and enabling transaction verification

UTXOs stands for unspent transaction outputs, which are the coins available to be spent. Fundamentally, there are two types of nodes, those that remain constantly connected to the network, known as **fully validating nodes**, and the so-called **lightweight nodes**. Fully validating nodes act as transaction validators, and in order to stay up to date and relay transactions across peers, they must maintain a local copy of the entire blockchain and have a reliable network connection. Ideally, the UTXO set is kept in RAM, so that when the node hears about a new proposed transaction on the network, it can quickly look up the transaction outputs being claimed, run the validation scripts, check the signatures, and, if valid, add the transaction to the mempool. (Sec 3.5 From Bitcoin Book)

Question 12

How does Bitcoin’s lock time feature work? Provide an example of how lock time can be used in a transaction.

Bitcoin’s **lock time** is a feature that allows a transaction to be pre-signed but not valid for inclusion in a block until a certain time or block height is reached.

Suppose Alice wants to pay Bob gradually for a service that she uses over time. Alice and Bob begin by creating a **multisig** transaction where the funds are locked in an output that requires both of their signatures to be spent. Alice funds this transaction with the maximum amount she might need to pay. Then, every minute, Alice creates a new transaction that spends the funds from the **multisig** output, sending Bob the amount owed up to that point and returning the rest to herself. For example:

- After one minute 1 BTC to Bob and 9 BTC back to Alice
- After two minutes 2 BTC to Bob and 8 BTC to Alice, and so on...

When Alice is done using the service, she informs Bob, who takes the latest transaction he received, signs it, and publishes it to the blockchain. This way, Bob gets paid for the service, and Alice receives the rest of her funds. However, if Bob refuses to cooperate and never signs the final transaction, the funds would remain locked in the **multisig** output, and

Alice wouldn't be able to recover them. To prevent this, before starting the protocol, Alice and Bob also jointly sign a refund transaction that returns all the funds to Alice, but with a lock time. If Bob refuses to sign any of Alice's micropayment transactions, she simply waits until the lock time expires and publishes the refund transaction to recover her money. (Sec 3.3 From Bitcoin Book)

Question 13

Describe how a micro-payment system can be implemented using Bitcoin scripts.

Suppose Alice wants to pay Bob gradually for a service that she uses over time. Alice and Bob begin by creating a **multisig** transaction where the funds are locked in an output that requires both of their signatures to be spent. Alice funds this transaction with the maximum amount she might need to pay. Then, every minute, Alice creates a new transaction that spends the funds from the **multisig** output, sending Bob the amount owed up to that point and returning the rest to herself. For example:

- After one minute 1 BTC to Bob and 9 BTC back to Alice
- After two minutes 2 BTC to Bob and 8 BTC to Alice, and so on...

When Alice is done using the service, she informs Bob, who takes the latest transaction he received, signs it, and publishes it to the blockchain. This way, Bob gets paid for the service, and Alice receives the rest of her funds. However, if Bob refuses to cooperate and never signs the final transaction, the funds would remain locked in the **multisig** output, and Alice wouldn't be able to recover them. To prevent this, before starting the protocol, Alice and Bob also jointly sign a refund transaction that returns all the funds to Alice, but with a lock time. If Bob refuses to sign any of Alice's micropayment transactions, she simply waits until the lock time expires and publishes the refund transaction to recover her money. (Sec 3.3 From Bitcoin Book)

Question 14

What is a Bitcoin multi-signature account, and how is it used?

A Bitcoin multi-signature account is a Bitcoin address that requires multiple private keys to authorize a transaction, basically a transaction is valid only if at least m out of n participants sign it. For example, a company with five partners and a Bitcoin reserve might want to protect its funds. To do so, an account can be created where transactions require the signatures of at least 3 out of the 5 partners. (Sec 4.3 From Bitcoin Book)

Question 15

What is a Bitcoin wallet? Explain the differences between custodial and non-custodial wallets and between hot and cold wallets

A Bitcoin wallet is a software that keeps track of your coins, manages your keys, and makes transactions easier through a user-friendly interface.

In a **custodial wallet**, a third party holds the private keys and is responsible for securing the funds (Binance for example), whereas in a **non-custodial wallet**, the user controls the keys and is fully responsible for the security of their assets.

The main difference between a **hot** and a **cold wallet** is that the former is connected to the internet, while the latter remains offline and is therefore generally more secure. Ideally, it's smart to use both, keeping the majority of funds in cold storage and only a smaller amount in a hot wallet for regular use. However, to do this safely, the two wallets must use different private keys, otherwise, if the hot wallet's private keys are compromised, the cold storage would be compromised too. Moreover, if the amount stored in the hot wallet grows too large, it can easily be transferred to the cold wallet without bringing it online. (Sec 4.1 and 4.2 From Bitcoin Book)

Question 16

What is slashing in Ethereum?

Slashing in Ethereum is a punitive mechanism designed to penalize validators who engage in harmful or dishonest behavior within the Proof-of-Stake consensus protocol. Penalties can range from slightly more than 0.5 ETH to the loss of the validator's entire stake, depending on the severity and context of the offense. Two main types of infractions can trigger slashing.

The first is known as **double proposal**, which occurs when a validator proposes more than one block for the same slot, violating the protocol's rules. A slot is a 12 seconds time interval during which a validator has the opportunity to propose a block. Slots are grouped in sets of 32 to form an epoch, meaning each epoch lasts approximately 6.4 minutes.

The second is called a **surround vote**, where a validator submits an FFG (Friendly Finality Gadget) vote that either surrounds or is surrounded by a previous vote they made, undermining the safety of the consensus process. An FFG is a vote cast by validators to determine the next check-

point block. If the vote reaches at least two-thirds of the total staked ETH, the checkpoints are updated, which means that the previous checkpoint becomes finalized, while the current checkpoint becomes justified. (<https://ethos.dev/beacon-chain#staking-rewards-and-penalties>)

Question 17

How is finality achieved in Ethereum under Proof of Stake?

Ethereum achieves finality under proof-of-stake through the Casper protocol. Basically, a transaction achieves finality when it is included in a block that cannot be reverted without burning a large amount of ETH. In the Casper protocol, finality is managed through checkpoint blocks. Specifically, the first block of each epoch is designated as a checkpoint. Validators cast votes for pairs of checkpoints they consider valid. If a pair receives votes representing at least two-thirds of the total staked ETH, the checkpoints are updated; the more recent one (the target) becomes justified, while the earlier one, already justified from a previous vote, is now upgraded to finalized.

(<https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/>)

Question 18

Explain the Ghost protocol and which is its goal.

The **GHOST** protocol was proposed as an alternative to Bitcoin's longest chain fork choice rule, with the goal of improving transaction throughput. Instead of selecting the longest chain, GHOST chooses the heaviest subtree, that is the branch with the most blocks in its descendants. This approach reflects the idea that a vote for a block implicitly supports all of its ancestors. Although Ethereum did not implement GHOST during its proof-of-work era, it later adopted a GHOST-inspired fork choice rule called LMD-GHOST after transitioning to proof-of-stake.

LMD-GHOST preserves the idea of voting, but has validators cast votes every 6.4 minutes on average, in order to provide updated information about the network's collective view of the chain. The "L" stands for latest, since only the most recent vote from each validator is taken into account. The "MD" stands for message-driven, meaning that votes are not embedded in blocks, but are instead explicit messages (attestations) sent by validators.

(https://eth2book.info/capella/part2/consensus/lmd_ghost/)

Question 19

Explain the Casper protocol and which is its goal.

Casper is Ethereum's proof-of-stake finality protocol. Its core objective is to guarantee that once a block is finalized, it becomes practically irreversible, unless an attacker is willing to lose a large amount of staked ETH. Casper achieves this by using a system of checkpoints: the first block of each epoch serves as a checkpoint, and validators regularly vote on pairs of checkpoints to confirm the chain's progress. When two-thirds of the total staked ETH supports a checkpoint, it is marked as justified, and if it builds on a previously justified checkpoint, that earlier one becomes finalized. (https://eth2book.info/capella/part2/consensus/lmd_ghost/)

Question 20

What are the main differences between a coin and a token in the blockchain ecosystem?

The main difference between these two classes of digital assets is that cryptocurrencies, or coins, are the native assets of a blockchain, while tokens are developed within a platform built on an existing blockchain. Thus, tokens are not part of a blockchain, instead, they are built on top of an existing blockchain through smart contracts. Among tokens, various types can be distinguished, including crypto tokens, NFTs (non-fungible tokens), and also security tokens. An example of a crypto token is Tether, which is a stablecoin operating on multiple blockchains, including Ethereum, and is pegged to the value of the US dollar, allowing the transfer of digital value without the typical volatility of cryptocurrencies. An additional difference is that cryptocurrency balances are tracked directly by the blockchain, whereas in the case of tokens, it is the smart contracts themselves that manage balances and provide transfer functions. Lastly, it's important to note that a blockchain can have only one native cryptocurrency, but it can host many different tokens.

Question 21

Explain how a Merkle tree works and its importance in blockchain systems.

A **Merkle tree** is a binary tree with hash pointers. Given a block, the leaves of the tree contain the hashes of the transactions, and each internal node contains the hash of the concatenation of its two child. The final result is the **Merkle root**, which is the hash at the top of the tree.

Only the Merkle root is stored in the block header. This allows, thanks to

the binary tree structure, to efficiently verify whether a specific transaction is part of the block (given n leaves, proof of membership in time $O(\log n)$) by simply considering the sibling hashes along the path from the transaction to the root. It also allows to detect tampering, since any tampering would alter the Merkle root. And finally, it allows the blockchain to store a single hash per block instead of one for every transaction, improving efficiency and saving space.

Optionally, the leaves of the Merkle tree can be pre-sorted, resulting in a **sorted Merkle tree**. With a sorted Merkle tree, it also becomes possible to efficiently prove non-membership. In this case, we can show that a particular transaction is not included in the block in logarithmic time and space. The idea is simple: we provide the path to the transaction that comes just before where the queried item would appear, and the path to the transaction that comes just after. If these two transactions are consecutive in the tree, this serves as a proof that the item in question is not part of the block. (Sec 1.2 From Bitcoin Book)

Question 22

What is the mempool in a blockchain network, and what role does it play in transaction processing?

Mempool stands for memory pool and is a component of fully validating nodes that holds all pending transactions. Since the network is peer-to-peer and not perfectly synchronized, each node maintains its own mempool, which is independent from the others. Mempools are used by miners to select transactions to include in a block, typically, they choose those with higher fees to maximize their profit. When a transaction is included in a block, it is removed from the mempool of all nodes that receive that block, as it is no longer pending. These storage areas also help partially prevent double spending, as they are designed to keep only the first valid transaction received for a given input and discard any conflicting ones. (Sec 2.2 From Bitcoin Book)

Question 23

What is a mining pool, and how is the work done by each participant estimated and rewarded?

Basically, a mining pool is a mutual insurance for Bitcoin miners. A group of miners will form a pool and all attempt to mine a block with a designated coinbase recipient called the **pool manager**. So, no matter who actually

finds the block, the pool manager will receive the rewards, will take that revenue, and distribute it to all the participants in the pool based on how much work each participant actually performed. To determine the proportions of the reward, miners can prove probabilistically how much work they're doing by outputting **shares**. Say the target is a number beginning with 67 zeros. A block's hash must be lower than the target for the block to be valid. In the process of searching for such a block, miners will find some blocks with hashes beginning with a lot of zeros, but not 67. Miners can show these nearly valid blocks to prove that they are indeed working.

The pool manager will also run a Bitcoin node, collecting transactions and assemble them into a block. The manager will include their own address in the coinbase transaction and send the block to all of the participants in the pool. All pool participants work on this block, and they prove that they've been working on it by sending in shares.

When a member of the pool finds a valid block, the pool manager distributes the reward in proportion to the amount of work done. The miner who actually finds the block is not awarded a special bonus, so if another miner did more work than, that other miner will be paid more even though they weren't the one who ended up finding a valid block.

Two simple methods of reward distribution based on shares are **pay-per-share** and **proportional**. In the PPS model, each miner is paid every time they submit a share, regardless of whether the pool eventually finds a valid block or not. This means the pool manager takes on most of the risk. In the Proportional model, the block reward is distributed based on the number of shares only after a valid block is found by the pool. (Sec 5.4 From Bitcoin Book)

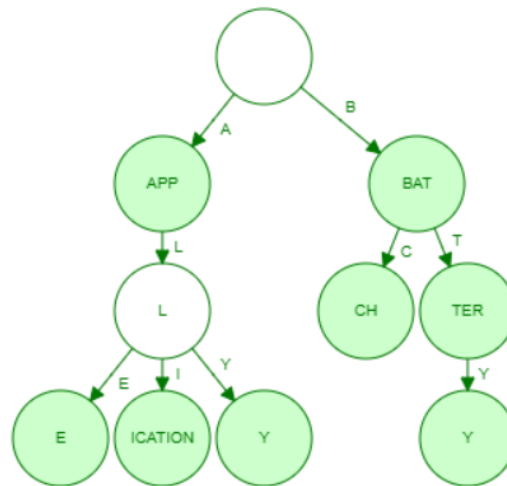
Question 24

Construct a radix tree for the following strings: apple, app, application, apply, bat, batch, batter, and battery.

Question 25

What is a Merkle-Patricia Trie, and how is it used in Ethereum?

The **Merkle Patricia Trie** is the data structure used to represent the state of Ethereum, which is the set of account, balances and smart contracts. A node in a Merkle Patricia Trie can be:



- **NULL** (an empty string)
- **branch** (a 17 item node $[v_0, \dots, v_{15}, vt]$)
- **leaf** (a 2 item node $[\text{encodedPath}, \text{value}]$)
- **extension** (a 2 item node $[\text{encodedPath}, \text{key}]$)

Moreover, every node is indexed by the hash of its contents. Let's take the following 4 addresses and their corresponding values as an example:

- a711355: 45.0 ETH
- a7f9365: 1.00 WEI
- a77d337: 1.10 ETH
- a77d387: 0.12 ETH

However, it's important to note that this is a simplification, since in Ethereum the paths are 64 characters long, or 32 bytes, as each character is represented by a nibble.

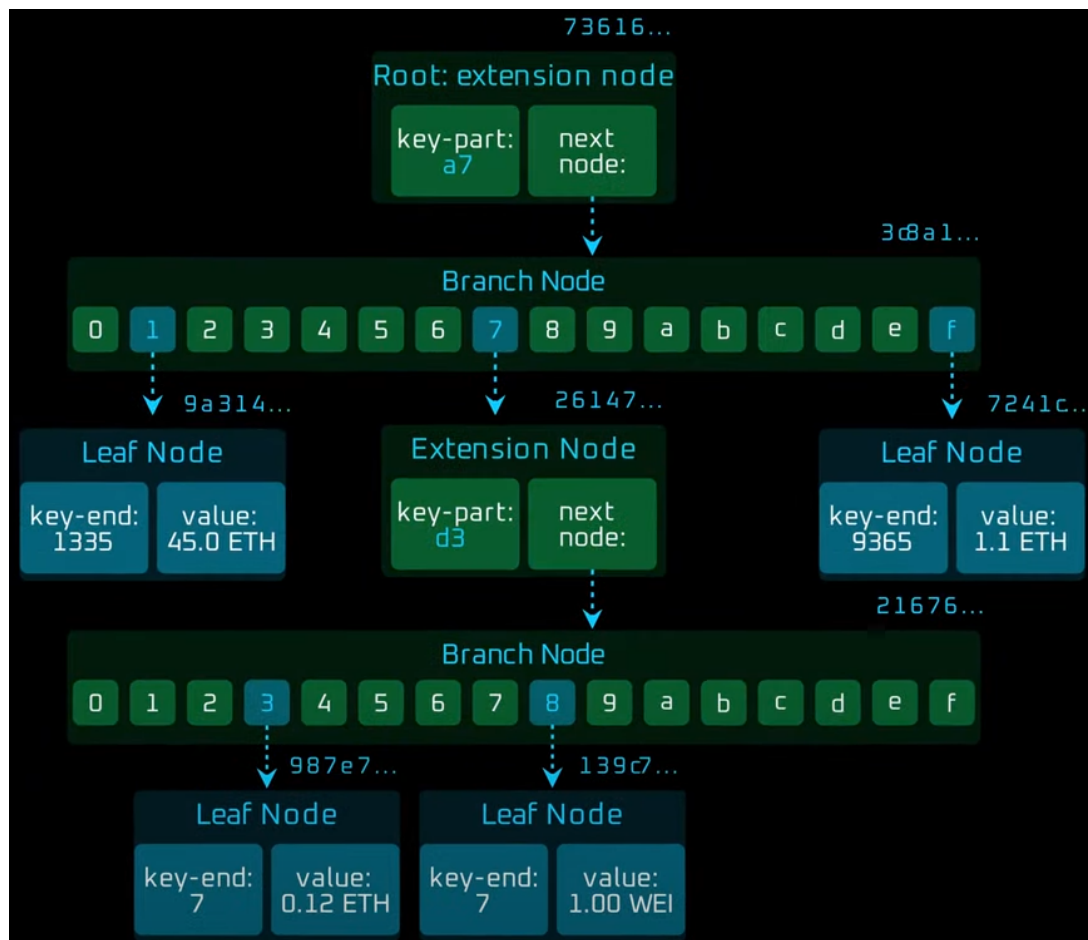
(ethereum.org/en/developers/docs/data-structures-and-encoding/patricia-merkle-trie/)

(<https://youtu.be/DGvRY9BjLRs>)

Question 26

What are the main differences between an Externally Owned Account (EOA) and a Contract Account in Ethereum?

Externally Owned Accounts and **Contract Accounts** are the two main types of Ethereum accounts, each with distinct characteristics. EOAs are user-controlled and secured by a public-private key pair. They are free to create and can initiate transactions directly. However, interactions between EOAs are limited to simple ETH or token transfers. In contrast, Contract Accounts are governed by smart contract code rather than private



keys. Deploying a contract account requires gas, as it uses network storage. Unlike EOAs, contract accounts cannot initiate transactions on their own, they can only respond to incoming transactions. When triggered, they can execute complex operations such as transferring tokens, modifying data, or even deploying new contracts.

(<https://ethereum.org/en/developers/docs/accounts/>)

Question 27

Describe how the block size in Ethereum is computed and its relationship with the gas fee.

Gas is the unit that measures the amount of computational effort required to execute specific operations on the Ethereum network. The **gas fee** is the amount of gas used to do some operation, and it ensures Ethereum is not vulnerable to spam and cannot get stuck in infinite computational loops. Gas prices are usually quoted in gwei, each gwei is equal to one-billionth of an ETH (0.000000001 ETH or 10^{-9} ETH). The total gas you pay is divided into two components, the **base fee** and the **priority fee**. The base fee is set by the protocol, which means you have to pay at least

this amount for your transaction to be considered valid. The **priority fee** is a tip that you add to the base fee to make your transaction attractive to validators.

Each block has a target size of 15 million gas, but the size of blocks will increase or decrease in accordance with network demand, up until the block limit of 30 million gas. The protocol achieves an equilibrium block size of 15 million on average through the following logic. If the block size is greater than the target block size, the protocol will increase the base fee for the following block. Similarly, the protocol will decrease the base fee if the block size is less than the target block size.

(<https://ethereum.org/en/developers/docs/gas/>)

Question 28

What are the two components of the gas fee in Ethereum? How are they computed, and who collects them?

Gas is the unit that measures the amount of computational effort required to execute specific operations on the Ethereum network. The **gas fee** is the amount of gas used to do some operation, and it ensures Ethereum is not vulnerable to spam and cannot get stuck in infinite computational loops. Gas prices are usually quoted in gwei, each gwei is equal to one-billionth of an ETH (0.000000001 ETH or 10^{-9} ETH). The total gas you pay is divided into two components, the **base fee** and the **priority fee**. The base fee is set by the protocol, which means you have to pay at least this amount for your transaction to be considered valid. The **priority fee** is a tip that you add to the base fee to make your transaction attractive to validators; without tips, validators could simply mine empty blocks, as they would receive the same block reward.

The base fee is determined by the previous block. When a new block is created, this base fee is burned, removing it from circulation. If the previous block was larger than the target size, the base fee increases by up to 12.5% for the next block; if it was smaller, the base fee decreases. This system helps maintain an average block size of 15 million gas.

(<https://ethereum.org/en/developers/docs/gas/>)

Question 29

Describe the lifecycle of an Ethereum validator, from activation to rewards and penalties.

To participate as a validator, a user must deposit 32 ETH into the deposit

contract and run three separate pieces of software: an execution client, a consensus client, and a validator client. On depositing their ETH, the user joins an activation queue that limits the rate of new validators joining the network. Once activated, validators receive new blocks from peers on the Ethereum network. The transactions delivered in the block are re-executed to check that the proposed changes to Ethereum's state are valid, and the block signature is checked. The validator then sends a vote (called an attestation) in favor of that block across the network.

One validator is randomly selected to be a block proposer in every slot. This validator is responsible for creating a new block and sending it out to other nodes on the network. Also in every slot, a committee of validators is randomly chosen, whose votes are used to determine the validity of the block being proposed.

Validators can be penalized for being inactive or missing attestations. More serious offenses, like proposing two blocks in the same slot or attesting to conflicting chains, result in slashing, which causes them to lose part or all of their staked ETH.

Validators can voluntarily exit the protocol by submitting an exit message. After exiting, there is a withdrawal delay before funds become available. (<https://ethereum.org/en/developers/docs/consensus-mechanisms/pos>)

Question 30

What is the inactivity leak in Ethereum's Proof of Stake, and how does it work?

The **inactivity leak** mechanism addresses the problem of how to recover finality in the event that over one-third of validators goes offline. Since finality requires a majority vote from validators representing 2/3 of the total stake.

The mechanism works as follows. When loss of finality is detected the inactivity leak gradually reduces the stakes of validators who are not making attestations until, eventually, the participating validators control 2/3 of the remaining stake. They can then begin to finalise checkpoints once again. It also provides a powerful incentive for stakers to fix any issues they have and to get back online.

(<https://eth2book.info/bellatrix/part2/incentives/inactivity/>)

Question 31

Describe the reward system in Ethereum's Proof of Stake (PoS) consensus mechanism.

Validators receive rewards when they make votes that are consistent with the majority of other validators, when they propose blocks, and when they participate in sync committees. The value of the rewards in each epoch are calculated from a `base_reward`, which is the base unit that other rewards are calculated from. The `base_reward` represents the average reward received by a validator under optimal conditions per epoch. This is calculated from the validator's effective balance and the total number of active validators as follows:

$$\text{base_reward} = \text{effective_balance} \frac{\text{base_reward_factor}}{\text{base_rew_per_epoch} \sqrt{\sum \text{active_balance}}}$$

where `base_reward_factor` is 64, `base_rewards_per_epoch` is 4 and the sum inside the square root is the total staked ether across all active validators. This means the base reward is proportional to the validator's effective balance and inversely proportional to the number of validators on the network.

The total reward is then calculated as the sum of five components that each have a weighting that determines how much each component adds to the total reward. The components are:

1. source vote: the validator has made a timely vote for the correct source checkpoint (weights 14)
2. target vote: the validator has made a timely vote for the correct target checkpoint (weights 26)
3. head vote: the validator has made a timely vote for the correct head block (weights 14)
4. sync committee reward: the validator has participated in a sync committee (weights 2)
5. proposer reward: the validator has proposed a block in the correct slot (weights 8)

These weights sum to 64. The reward is then calculated as the sum of the applicable weights divided by 64. Since a validator usually is not a block proposer, their maximum reward is:

$$64 - 8/64 * \text{base_reward} == 7/8 * \text{base_reward}$$

Validators that are neither block proposers nor in a sync committee can receive:

$$64 - 8 - 2/64 * \text{base_reward} == 6.75/8 * \text{base_reward}$$

(<https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/rewards-and-penalties/>)

Question 32

What are the key differences between a centralized exchange (CEX) and a decentralized exchange (DEX)?

Centralized exchanges are controlled by a single entity, offering high user-friendliness, fast transactions, and broad fiat support, but they require users to give up custody of their funds and typically offer lower privacy and transparency. They're more widely adopted but can be vulnerable to hacks and operational risks.

Decentralized exchanges (DEXs) run on distributed systems, giving users full control over their assets, with higher privacy and transparency. However, they are often less user-friendly, have slower transactions, lower fiat support, and variable liquidity. Adoption is growing but remains lower than CEXs. (Pump and Dump)

Question 33

Describe how a pump and dump operation works on CEX.

A pump and dump operation on a centralized exchange (CEX) is a market manipulation scheme where a group of traders artificially inflates the price of a cryptocurrency (the "pump") and then quickly sells it at a profit (the "dump"), leaving others with losses.

Chat groups or forums are used to announce a market operation, often specifying an exact time and sometimes even including a countdown. The goal is to attract outside investors, sometimes through the spread of fake news, in order to push a large number of people to move the market according to the admins' strategy. In the end, however, only the admins make a profit, while the others are left with losses. (Pump and Dump)

Question 34

What is wash trading in cryptocurrency markets?

Wash trading is a form of market manipulation in which a set of colluding users trade the same asset to create artificial activity. On the stock market, wash trading is usually performed by simultaneously placing a sell and buy order on the same asset without incurring market risk or changing the trader's market position. The purpose of wash trading is to influence a

specific asset's pricing or trading activities, generating interest in the asset and attracting external traders. This practice has been illegal for almost a century in the U.S. stock markets, however, in the unregulated crypto market, this is not true, and in the NFT community, people believe wash trading activity significantly impacts NFTs' trading volume. In the NFT ecosystem, the main concern is that wash trading can create the illusion that there is market interest in the target NFTs. The goal is to increase their value artificially. Furthermore, since some NFTMs (NFT Marketplaces) reward users according to their trading volumes, wash trading can also be used to claim more undeserved reward tokens.

(Paper "A game of NFTs")

Question 35

What is a liquidity pool in decentralized finance (DeFi), what is its purpose, and how does it work?

A liquidity pool in decentralized finance, is a collection of funds locked into a smart contract. These funds are provided by users known as liquidity providers, who deposit pairs of tokens into the pool. This setup allows other users to trade between those tokens directly through the pool, without needing a traditional buyer or seller on the other side of the transaction.

The primary purpose of a liquidity pool is to ensure continuous and automated access to liquidity. In traditional finance, trading typically requires a matching engine to pair buyers and sellers, which can lead to delays or failures in execution if one side of the trade is missing. DeFi bypasses this need by allowing trades to occur directly against the pool, which always holds reserves of the token pair.

This mechanism not only allows for fast and decentralized trading but also opens up passive income opportunities for liquidity providers, who earn a portion of the transaction fees generated by the pool. However, it's important to note that providing liquidity is not risk-free; participants are exposed to phenomena like impermanent loss, where the value of their deposited assets can diverge unfavorably due to market fluctuations.

(Liquidity Pool and Rug Pull)

Question 36

What are the basic steps involved in performing a rug pull?

The scam usually begins with (1) the creation of a new token, often hyped up with a flashy website, social media campaigns, and exaggerated

promises of returns. The developers then launch the token on a DEX, (2) creating a liquidity pool where users can trade the token against a well-known asset like ETH for example, and of course, (3) add liquidity. Then early buyers join in, and (4) the pool grows as more users provide liquidity or buy the token, driving up its price. Once the pool has attracted enough real liquidity, the developers (5) withdraw all the liquidity from the pool, effectively removing the ability to trade the token. After that, the scammers typically vanish, deleting websites, shutting down social media, and leaving investors with worthless tokens. (Liquidity Pool and Rug Pull)